

## Required Libraries

```
pip install numpy pillow matplotlib
```

---

### Task 1: Load an Image and Convert it to a NumPy Array

```
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt

# Load Image
image = Image.open("virat_kohli.jpg")    # Replace with your image path

# Convert to NumPy Array
image_array = np.array(image)

# Print Information
print("Image Shape:", image_array.shape)
print("Data Type:", image_array.dtype)

# Display Image
plt.imshow(image_array)
plt.title("Sample Image")
plt.axis("off")
plt.show()
```

### Example Output

Image Shape: (600, 800, 3)

Data Type: uint8

### Explanation

If the shape is:

(600, 800, 3)

It means:

- **600** → Height
- **800** → Width
- **3** → RGB Channels

CNNs process images as multidimensional NumPy arrays.

---

## Task 2: Implement 2D Convolution Using NumPy

### Input Image (5×5)

```
import numpy as np
```

```
image = np.array([
    [1,2,3,0,1],
    [4,5,6,1,2],
    [7,8,9,2,3],
    [1,2,3,4,5],
    [6,7,8,9,0]
])
```

### Filter (Kernel)

```
kernel = np.array([
    [1,0,-1],
    [1,0,-1],
    [1,0,-1]
])
```

### Convolution Code

```
output = np.zeros((3,3))

for i in range(3):

    for j in range(3):

        region = image[i:i+3, j:j+3]

        output[i,j] = np.sum(region * kernel)

print(output)
```

### Example Output

```
[[ -6. -6.  6.]
 [ -6. -6.  6.]
 [ -6. -6. 12.]]
```

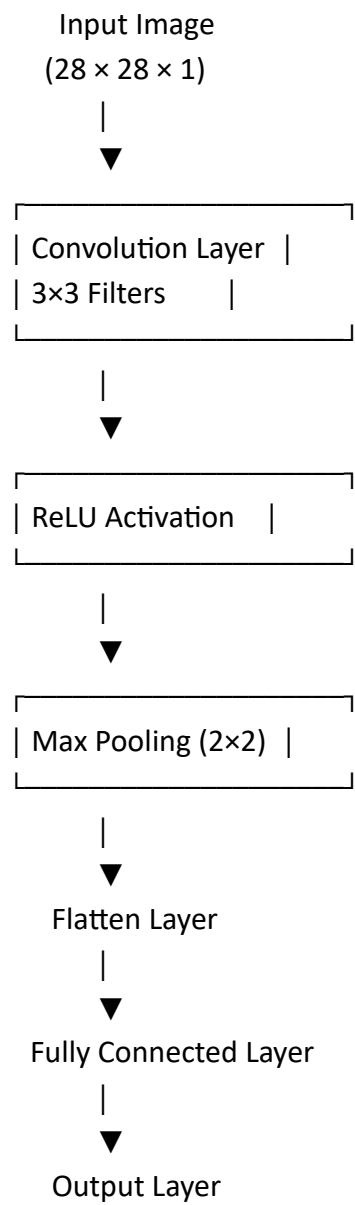
### Explanation

The 3×3 kernel slides over the image, multiplies corresponding values, sums them, and stores the result in the output feature map.

---

**Task 3: Basic CNN Architecture**

**Diagram**



**Layer Functions**

| Layer       | Function                            |
|-------------|-------------------------------------|
| Input Layer | Receives the image as pixel values. |

| Layer                    | Function                                                                      |
|--------------------------|-------------------------------------------------------------------------------|
| <b>Convolution Layer</b> | Detects edges, corners, textures, and patterns using filters.                 |
| <b>ReLU Layer</b>        | Introduces non-linearity and helps the network learn complex features.        |
| <b>Pooling Layer</b>     | Reduces image size while preserving important features, improving efficiency. |
| <b>Flatten Layer</b>     | Converts the 2D feature maps into a 1D vector for classification.             |
| <b>Output Layer</b>      | Produces the final class probabilities (e.g., cat, dog, digit).               |

---

#### Task 4: Max Pooling (Without TensorFlow/PyTorch)

```
import numpy as np
```

```
def max_pooling(image):
```

```
    rows, cols = image.shape
```

```
    output_rows = rows // 2
```

```
    output_cols = cols // 2
```

```
    pooled = np.zeros((output_rows, output_cols))
```

```
    for i in range(output_rows):
```

```
        for j in range(output_cols):
```

```
            block = image[
                i*2:i*2+2,
                j*2:j*2+2
            ]
```

```
            pooled[i,j] = np.max(block)
```

```
    return pooled
```

```
image = np.array([
```

```

    [1,2,3,4],
    [5,6,7,8],
    [2,4,6,8],
    [1,3,5,7]
])

result = max_pooling(image)

print("Original Image")

print(image)

print()

print("After Max Pooling")

print(result)

```

### Output

Original Image

```

[[1 2 3 4]
 [5 6 7 8]
 [2 4 6 8]
 [1 3 5 7]]

```

After Max Pooling

```

[[6. 8.]
 [4. 8.]]

```

### Explanation

Each **2×2 block** is replaced with its **maximum value**, reducing the spatial dimensions while keeping the strongest feature.

## Task 5: Translation Invariance in CNNs

### Instagram Example

Instagram uses CNNs to recognize faces in photos. Whether a person's face appears in the center, left, or right side of an image, the CNN can still detect and recognize it because the

learned filters respond to the same facial features regardless of their position. This property is known as **translation invariance**.

### Zomato Example

Zomato can identify food items from uploaded images even if the food is placed in different parts of the photo. A pizza or burger may appear at the top, bottom, or corner of the image, but the CNN still recognizes it by detecting its characteristic visual features. This makes food recognition reliable despite changes in object position.

---

### Summary

| Task Concept             | Key Learning                                                                                                                                                                  |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 Image as NumPy Array   | Images are represented as arrays with shape (height, width, channels) for CNN processing.                                                                                     |
| 2 Manual 2D Convolution  | A kernel slides across the image, computing weighted sums to create a feature map.                                                                                            |
| 3 CNN Architecture       | Convolution extracts features, ReLU adds non-linearity, pooling reduces size, and the output layer performs classification.                                                   |
| 4 Max Pooling            | Replaces each 2×2 region with its maximum value to retain important features while reducing dimensions.                                                                       |
| 5 Translation Invariance | CNNs recognize the same object even when it appears in different positions within an image, enabling robust applications like face recognition and food image classification. |